

```

import { useState } from "react";

const ACCESS_CODE = "46587949";

function LoginScreen({ onAccess }) {
  const [code, setCode] = useState("");
  const [error, setError] = useState(false);
  const [shake, setShake] = useState(false);

  const handleSubmit = () => {
    if (code === ACCESS_CODE) {
      onAccess();
    } else {
      setError(true);
      setShake(true);
      setTimeout(() => setShake(false), 600);
    }
  };

  return (
    <div style={{
      minHeight: "100vh", background: "#0f0f0f",
      display: "flex", alignItems: "center", justifyContent: "center",
      fontFamily: "'DM Sans', sans-serif"
    }}>
      <style>{`
        @import url('https://fonts.googleapis.com/css2?family=Playfair+Display:wght@400,500,600,700,800,900&display=block');
        @keyframes shake {
          0%,100%{transform:translateX(0)} 20%{transform:translateX(-8px)} 40%{transform:translateX(8px)} 60%{transform:translateX(-8px)} 80%{transform:translateX(8px)}
        }
        .shake { animation: shake 0.5s ease; }
        .login-btn:hover { opacity: 0.85; }
        .login-input:focus { border-color: #c9a84c88 !important; outline: none; }
      `}</style>
      <div style={{ width: "100%", maxWidth: 400, padding: "0 24px" }}>
        <div style={{ textAlign: "center", marginBottom: 40 }}>
          <div style={{
            width: 56, height: 56, background: "linear-gradient(135deg,#c9a84c,#808080)",
            borderRadius: 14, display: "flex", alignItems: "center", justifyConte
            margin: "0 auto 20px", fontSize: 26, fontWeight: 700,
            fontFamily: "'Playfair Display', serif", color: "#fff"
          }}>R</div>
          <div style={{ fontFamily: "'Playfair Display', serif", fontSize: 28, fo
          <div style={{ fontSize: 14, color: "#6a5a3a", fontWeight: 300 }}>Anális

```

```

</div>
<div className={shake ? "shake" : ""} style={{
  background: "#161410", border: `1px solid ${error ? "#6a2020" : "#2a201"}
  borderRadius: 16, padding: "32px 28px", transition: "border-color 0.3s"
}}>
  <div style={{ fontSize: 13, color: "#6a5a3a", marginBottom: 8, fontWeig
    Código de acceso
  </div>
  <input
    className="login-input"
    type="password"
    value={code}
    onChange={e => { setCode(e.target.value); setError(false); }}
    onKeyDown={e => e.key === "Enter" && handleSubmit()}
    placeholder="....."
    style={{
      width: "100%", background: "#0f0f0f",
      border: `1px solid ${error ? "#6a2020" : "#2a2010"}`,
      borderRadius: 10, padding: "13px 16px",
      color: "#f0ece4", fontSize: 18, letterSpacing: 6,
      fontFamily: "'DM Sans', sans-serif", marginBottom: 8,
      transition: "border-color 0.2s", boxSizing: "border-box"
    }}
    autoFocus
  />
  {error && <div style={{ fontSize: 12, color: "#c06060", marginBottom: 1
  <button className="login-btn" onClick={handleSubmit} style={{
    width: "100%", background: "linear-gradient(135deg,#c9a84c,#8b6914)",
    color: "#fff", border: "none", borderRadius: 10,
    padding: "13px", fontSize: 15, fontWeight: 600,
    fontFamily: "'DM Sans', sans-serif", cursor: "pointer",
    marginTop: error ? 0 : 8, transition: "opacity 0.2s"
  }}>
    Ingresar →
  </button>
</div>
<div style={{ textAlign: "center", marginTop: 20, fontSize: 12, color: "#
  Acceso restringido · Beta privada
</div>
</div>
</div>
);
}

```

```

const PROP_TYPES = ["Casa", "PH", "Departamento", "Terreno", "Local", "Oficina"];
const CONDITIONS = ["Excelente", "Muy bueno", "Bueno", "Regular", "A refaccionar"

```

```

const emptySubject = {
  direccion: "", tipo: "Casa", m2_totales: "", m2_cubiertos: "",
  ambientes: "", dormitorios: "", banos: "", cochera: false,
  piscina: false, antigüedad: "", estado: "Bueno", notas: ""
};

const emptyComp = {
  direccion: "", tipo: "Casa", m2_totales: "", m2_cubiertos: "",
  ambientes: "", dormitorios: "", banos: "", cochera: false,
  piscina: false, antigüedad: "", estado: "Bueno", precio: "", moneda: "USD"
};

export default function CMAApp() {
  const [authenticated, setAuthenticated] = useState(false);
  const [step, setStep] = useState(1);
  const [subject, setSubject] = useState(emptySubject);
  const [comps, setComps] = useState([{}]);
  const [pasteText, setPasteText] = useState("");
  const [inputMode, setInputMode] = useState("form");
  const [loading, setLoading] = useState(false);
  const [report, setReport] = useState(null);
  const [error, setError] = useState("");
  const [parsedFromText, setParsedFromText] = useState(false);

  const updateSubject = (k, v) => setSubject(p => ({ ...p, [k]: v }));
  const updateComp = (i, k, v) => setComps(p => p.map((c, idx) => idx === i ? {
  const addComp = () => setComps(p => [...p, {}]);
  const removeComp = (i) => setComps(p => p.filter((_, idx) => idx !== i));

  const parseTextComps = async () => {
    if (!pasteText.trim()) return;
    setLoading(true);
    setError("");
    try {
      const res = await fetch("https://api.anthropic.com/v1/messages", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          model: "claude-sonnet-4-20250514",
          max_tokens: 1000,
          system: `Eres un asistente para análisis inmobiliario. El usuario te pe
Extrae cada propiedad y devuelve SOLO un JSON array (sin backticks, sin texto ext
direccion, tipo (Casa/PH/Departamento/Terreno/Local/Oficina), m2_totales, m2_cubi
Si algún campo no está disponible usa "" o false según corresponda.`
          messages: [{ role: "user", content: pasteText }]
        })
      });
    }
  };
}

```

```

    const data = await res.json();
    const text = data.content.map(b => b.text || "").join("");
    const parsed = JSON.parse(text.replace(/``json|``/g, "").trim());
    setComps(parsed.map(p => ({ ...emptyComp, ...p })));
    setParsedFromText(true);
    setInputMode("form");
  } catch (e) {
    setError("No se pudo interpretar el texto. Revisá el formato e intentá de n
  }
  setLoading(false);
};

```

```

const generateReport = async () => {
  setLoading(true);
  setError("");
  setReport(null);
  try {
    const prompt = `Eres un analista inmobiliario experto. Generá un Análisis C

```

PROPIEDAD SUJETO:

`${JSON.stringify(subject, null, 2)}`

PROPIEDADES COMPARABLES:

`${JSON.stringify(comps, null, 2)}`

Generá el análisis con las siguientes secciones, usando exactamente estos marcado

[RESUMEN]

Breve descripción de la propiedad sujeto y el contexto del análisis.

[TABLA_COMPARATIVA]

Tabla comparando la propiedad sujeto con cada comparable, destacando diferencias

[AJUSTES]

Para cada comparable, explicá los ajustes necesarios (positivos o negativos) por

[VALOR_ESTIMADO]

Rango de valor estimado de la propiedad sujeto en USD, con justificación basada e

[CONCLUSION]

Recomendación profesional: precio de publicación sugerido, observaciones del merc

```

const res = await fetch("https://api.anthropic.com/v1/messages", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    model: "claude-sonnet-4-20250514",

```

```

        max_tokens: 1000,
        messages: [{ role: "user", content: prompt }]
    })
});
const data = await res.json();
const text = data.content.map(b => b.text || "").join("");
setReport(text);
setStep(3);
} catch (e) {
    setError("Error al generar el análisis. Intentá de nuevo.");
}
setLoading(false);
};

const parseReport = (text) => {
    const sections = {};
    const markers = ["RESUMEN", "TABLA_COMPARATIVA", "AJUSTES", "VALOR_ESTIMADO",
    markers.forEach((m, i) => {
        const start = text.indexOf(`[${m}]`);
        const next = markers[i + 1] ? text.indexOf(`[${markers[i + 1]}]`) : text.le
        if (start !== -1) sections[m] = text.slice(start + m.length + 2, next).trim
    });
    return sections;
};

const FieldInput = ({ label, value, onChange, type = "text", small = false }) =
    <div className={small ? "field-sm" : "field"}>
        <label>{label}</label>
        <input type={type} value={value} onChange={e => onChange(e.target.value)} /
    </div>
);

const SelectInput = ({ label, value, onChange, options }) => (
    <div className="field">
        <label>{label}</label>
        <select value={value} onChange={e => onChange(e.target.value)}>
            {options.map(o => <option key={o}>{o}</option>)}
        </select>
    </div>
);

const Toggle = ({ label, value, onChange }) => (
    <div className="toggle-field">
        <span>{label}</span>
        <button className={`toggle ${value ? "on" : ""}`} onClick={() => onChange(!
            {value ? "Sí" : "No"}
        </button>

```

```

    </div>
  );

const sections = report ? parseReport(report) : {};

if (!authenticated) return <LoginScreen onAccess={() => setAuthenticated(true)}

return (
  <div className="app">
    <style>{`
      @import url('https://fonts.googleapis.com/css2?family=Playfair+Display:wg

      *, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }

      .app {
        min-height: 100vh;
        background: #0f0f0f;
        color: #f0ece4;
        font-family: 'DM Sans', sans-serif;
        padding: 0 0 60px;
      }

      .header {
        background: linear-gradient(135deg, #1a1208 0%, #0f0f0f 60%);
        border-bottom: 1px solid #2a2010;
        padding: 28px 40px 24px;
        display: flex;
        align-items: center;
        gap: 16px;
      }

      .header-logo {
        width: 42px; height: 42px;
        background: linear-gradient(135deg, #c9a84c, #8b6914);
        border-radius: 8px;
        display: flex; align-items: center; justify-content: center;
        font-size: 20px; font-weight: 700; color: #fff;
        font-family: 'Playfair Display', serif;
      }

      .header-text h1 {
        font-family: 'Playfair Display', serif;
        font-size: 22px; font-weight: 700;
        color: #f0ece4; letter-spacing: -0.3px;
      }

      .header-text p {
        font-size: 12px; color: #8a7a5a; margin-top: 2px; font-weight: 300;
      }

      .badge {

```

```

    margin-left: auto;
    background: #c9a84c22;
    border: 1px solid #c9a84c44;
    color: #c9a84c;
    font-size: 11px; font-weight: 600;
    padding: 4px 10px; border-radius: 20px;
    letter-spacing: 0.5px;
}

.steps {
    display: flex; align-items: center; gap: 0;
    padding: 24px 40px 0;
    max-width: 860px; margin: 0 auto;
}

.step-item {
    display: flex; align-items: center; gap: 8px;
    font-size: 13px; font-weight: 500;
    color: #4a4030; cursor: default;
    flex: 1;
}

.step-item.active { color: #c9a84c; }
.step-item.done { color: #7a9a5a; cursor: pointer; }
.step-num {
    width: 28px; height: 28px; border-radius: 50%;
    border: 2px solid #2a2010;
    display: flex; align-items: center; justify-content: center;
    font-size: 12px; font-weight: 700; flex-shrink: 0;
}

.step-item.active .step-num { border-color: #c9a84c; color: #c9a84c; back
.step-item.done .step-num { border-color: #7a9a5a; color: #7a9a5a; backgr
.step-line { flex: 1; height: 1px; background: #2a2010; margin: 0 8px; }

.content { max-width: 860px; margin: 32px auto 0; padding: 0 40px; }

.section-title {
    font-family: 'Playfair Display', serif;
    font-size: 26px; font-weight: 700;
    color: #f0ece4; margin-bottom: 6px;
}

.section-sub { font-size: 14px; color: #6a5a3a; margin-bottom: 28px; font

.card {
    background: #161410;
    border: 1px solid #2a2010;
    border-radius: 12px;
    padding: 24px;
    margin-bottom: 16px;

```

```

}
.card-title {
  font-size: 13px; font-weight: 600; color: #c9a84c;
  text-transform: uppercase; letter-spacing: 1px;
  margin-bottom: 18px;
}

.fields-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 14px; }
.fields-grid-3 { display: grid; grid-template-columns: 1fr 1fr 1fr; gap:

.field { display: flex; flex-direction: column; gap: 6px; }
.field-sm { display: flex; flex-direction: column; gap: 6px; }
.field label, .field-sm label {
  font-size: 11px; font-weight: 600; color: #6a5a3a;
  text-transform: uppercase; letter-spacing: 0.6px;
}
.field input, .field select, .field textarea,
.field-sm input, .field-sm select {
  background: #0f0f0f; border: 1px solid #2a2010;
  border-radius: 8px; padding: 10px 12px;
  color: #f0ece4; font-family: 'DM Sans', sans-serif;
  font-size: 14px; outline: none;
  transition: border-color 0.2s;
}
.field input:focus, .field select:focus, .field textarea:focus,
.field-sm input:focus { border-color: #c9a84c66; }
.field textarea { resize: vertical; min-height: 70px; }
.field select option { background: #1a1208; }

.toggles-row { display: flex; gap: 16px; margin-top: 6px; }
.toggle-field {
  display: flex; align-items: center; gap: 10px;
  font-size: 13px; color: #8a7a5a;
}
.toggle {
  background: #1a1208; border: 1px solid #2a2010;
  color: #4a4030; font-size: 12px; font-weight: 600;
  padding: 5px 14px; border-radius: 20px; cursor: pointer;
  transition: all 0.2s;
}
.toggle.on {
  background: #c9a84c22; border-color: #c9a84c66;
  color: #c9a84c;
}

.comp-header {
  display: flex; align-items: center; justify-content: space-between;

```



```

    margin-bottom: 16px;
}
.comp-label {
    font-size: 12px; font-weight: 700; color: #c9a84c;
    text-transform: uppercase; letter-spacing: 1px;
}
.btn-remove {
    background: none; border: 1px solid #3a2020;
    color: #7a4040; font-size: 12px; padding: 4px 10px;
    border-radius: 6px; cursor: pointer;
    transition: all 0.2s;
}
.btn-remove:hover { background: #3a202022; color: #c06060; }

.price-row { display: grid; grid-template-columns: 80px 1fr; gap: 10px; }
.price-row select { padding: 10px 8px; }

.mode-tabs {
    display: flex; gap: 0; border: 1px solid #2a2010;
    border-radius: 10px; overflow: hidden; margin-bottom: 20px;
    width: fit-content;
}
.mode-tab {
    padding: 9px 20px; font-size: 13px; font-weight: 500;
    background: #0f0f0f; color: #4a4030;
    border: none; cursor: pointer; transition: all 0.2s;
}
.mode-tab.active {
    background: #c9a84c22; color: #c9a84c;
}

textarea.paste-area {
    width: 100%; background: #0f0f0f;
    border: 1px solid #2a2010; border-radius: 10px;
    padding: 16px; color: #f0ece4;
    font-family: 'DM Sans', sans-serif; font-size: 13px;
    min-height: 160px; outline: none; resize: vertical;
    line-height: 1.6;
}
textarea.paste-area:focus { border-color: #c9a84c44; }
.paste-hint {
    font-size: 12px; color: #4a4030; margin-top: 8px; font-style: italic;
}

.btn-primary {
    background: linear-gradient(135deg, #c9a84c, #8b6914);
    color: #fff; font-family: 'DM Sans', sans-serif;

```

```
    font-size: 14px; font-weight: 600;
    padding: 13px 28px; border-radius: 10px;
    border: none; cursor: pointer;
    transition: opacity 0.2s; letter-spacing: 0.2px;
}
.btn-primary:hover { opacity: 0.88; }
.btn-primary:disabled { opacity: 0.4; cursor: not-allowed; }

.btn-secondary {
    background: #161410; color: #c9a84c;
    border: 1px solid #c9a84c44;
    font-family: 'DM Sans', sans-serif;
    font-size: 14px; font-weight: 500;
    padding: 12px 24px; border-radius: 10px;
    cursor: pointer; transition: all 0.2s;
}
.btn-secondary:hover { background: #c9a84c11; }

.btn-add {
    background: none; border: 1px dashed #2a2010;
    color: #6a5a3a; font-family: 'DM Sans', sans-serif;
    font-size: 13px; font-weight: 500;
    padding: 12px; border-radius: 10px;
    cursor: pointer; width: 100%;
    transition: all 0.2s; margin-top: 8px;
}
.btn-add:hover { border-color: #c9a84c44; color: #c9a84c; }

.actions-row {
    display: flex; gap: 12px; align-items: center;
    margin-top: 28px; flex-wrap: wrap;
}

.error-box {
    background: #2a0f0f; border: 1px solid #6a2020;
    border-radius: 8px; padding: 12px 16px;
    color: #e08080; font-size: 13px; margin-top: 16px;
}

.loading-overlay {
    display: flex; flex-direction: column;
    align-items: center; justify-content: center;
    padding: 60px 0; gap: 16px;
}

.spinner {
    width: 40px; height: 40px;
    border: 3px solid #2a2010;
```

```

    border-top-color: #c9a84c;
    border-radius: 50%;
    animation: spin 0.8s linear infinite;
}
@keyframes spin { to { transform: rotate(360deg); } }
.loading-text { font-size: 14px; color: #6a5a3a; }

.report-section {
    background: #161410; border: 1px solid #2a2010;
    border-radius: 12px; margin-bottom: 12px; overflow: hidden;
}
.report-section-header {
    padding: 14px 20px;
    display: flex; align-items: center; gap: 10px;
    border-bottom: 1px solid #2a2010;
}
.report-section-icon {
    width: 28px; height: 28px;
    border-radius: 6px; display: flex;
    align-items: center; justify-content: center; font-size: 14px;
}
.report-section-title {
    font-size: 13px; font-weight: 700; color: #f0ece4;
    text-transform: uppercase; letter-spacing: 0.8px;
}
.report-section-body {
    padding: 20px; font-size: 14px; color: #c0b090;
    line-height: 1.75; white-space: pre-wrap;
}

.highlight-box {
    background: linear-gradient(135deg, #c9a84c11, #8b691411);
    border: 1px solid #c9a84c33;
    border-radius: 12px; padding: 24px; margin-bottom: 20px;
    text-align: center;
}
.highlight-box .hl-label {
    font-size: 11px; color: #8a7a5a; text-transform: uppercase;
    letter-spacing: 1px; margin-bottom: 8px;
}
.highlight-box .hl-value {
    font-family: 'Playfair Display', serif;
    font-size: 32px; font-weight: 700; color: #c9a84c;
}

.parsed-badge {
    background: #1a2a10; border: 1px solid #4a7a2a44;

```

```

        color: #8aba5a; font-size: 12px; padding: 6px 14px;
        border-radius: 20px; margin-bottom: 16px; display: inline-flex;
        align-items: center; gap: 6px;
    }

    @media (max-width: 600px) {
        .header { padding: 20px; }
        .content { padding: 0 16px; }
        .steps { padding: 20px 16px 0; }
        .fields-grid, .fields-grid-3 { grid-template-columns: 1fr; }
    }
`</style>

<div className="header">
    <div className="header-logo">R</div>
    <div className="header-text">
        <h1>Análisis Comparativo de Mercado</h1>
        <p>Herramienta CMA con inteligencia artificial · RE/MAX</p>
    </div>
    <span className="badge">♦ IA</span>
</div>

<div className="steps">
    {[
        { n: 1, label: "Propiedad sujeto" },
        { n: 2, label: "Comparables" },
        { n: 3, label: "Análisis IA" }
    ].map((s, i) => (
        <div key={s.n} style={{ display: "flex", alignItems: "center", flex: 1 }}>
            <div
                className={`step-item ${step === s.n ? "active" : step > s.n ? "done" : ""}
                onClick={() => step > s.n && setStep(s.n)}
            >
                <div className="step-num">{step > s.n ? "✓" : s.n}</div>
                <span>{s.label}</span>
            </div>
            {i < 2 && <div className="step-line" />}
        </div>
    )))
</div>

<div className="content">
    {/* STEP 1 */}
    {step === 1 && (
        <>
            <p className="section-title" style={{ marginTop: 32 }}>Propiedad suje
            <p className="section-sub">Cargá los datos de la propiedad que querés

```

```

<div className="card">
  <div className="card-title">Datos generales</div>
  <div className="fields-grid" style={{ marginBottom: 14 }}>
    <div className="field" style={{ gridColumn: "span 2" }}>
      <label>Dirección</label>
      <input value={subject.direccion} onChange={e => updateSubject("
    </div>
  </div>
  <div className="fields-grid-3" style={{ marginBottom: 14 }}>
    <SelectInput label="Tipo" value={subject.tipo} onChange={v => upd
    <SelectInput label="Estado" value={subject.estado} onChange={v =>
    <FieldInput label="Antigüedad (años)" value={subject.antigüedad}
  </div>
  <div className="fields-grid-3" style={{ marginBottom: 14 }}>
    <FieldInput label="M² totales" value={subject.m2_totales} onChang
    <FieldInput label="M² cubiertos" value={subject.m2_cubiertos} onC
    <FieldInput label="Ambientes" value={subject.ambientes} onChange=
  </div>
  <div className="fields-grid-3" style={{ marginBottom: 14 }}>
    <FieldInput label="Dormitorios" value={subject.dormitorios} onCha
    <FieldInput label="Baños" value={subject.banos} onChange={v => up
    <div />
  </div>
  <div className="toggles-row">
    <Toggle label="Cochera" value={subject.cochera} onChange={v => up
    <Toggle label="Piscina" value={subject.piscina} onChange={v => up
  </div>
</div>
<div className="card">
  <div className="card-title">Observaciones</div>
  <div className="field">
    <label>Notas adicionales</label>
    <textarea value={subject.notas} onChange={e => updateSubject("not
  </div>
</div>
<div className="actions-row">
  <button className="btn-primary" onClick={() => setStep(2)} disabled
    Continuar →
  </button>
</div>
</>
)}

{/* STEP 2 */}
{step === 2 && (
  <>
    <p className="section-title" style={{ marginTop: 32 }}>Propiedades co

```

```
<p className="section-sub">Agregá las propiedades similares de la zon
```

```
<div className="mode-tabs">
```

```
  <button className={`mode-tab ${inputMode === "form" ? "active" : ""}
```

```
  <button className={`mode-tab ${inputMode === "paste" ? "active" : ""}
```

```
</div>
```

```
{inputMode === "paste" && (
```

```
  <div className="card">
```

```
    <div className="card-title">Pegar propiedades</div>
```

```
    <textarea
```

```
      className="paste-area"
```

```
      value={pasteText}
```

```
      onChange={e => setPasteText(e.target.value)}
```

```
      placeholder={`Pegá aquí cualquier texto con las propiedades. Po  
/>
```

```
    <p className="paste-hint">La IA extrae los datos automáticamente.
```

```
    <div className="actions-row">
```

```
      <button className="btn-primary" onClick={parseTextComps} disabl  
        {loading ? "Procesando..." : "Extraer con IA →"}
```

```
    </button>
```

```
  </div>
```

```
</div>
```

```
)}
```

```
{inputMode === "form" && (
```

```
  <>
```

```
    {parsedFromText && (
```

```
      <div className="parsed-badge">✓ {comps.length} propiedades extr  
    )}
```

```
    {comps.map((comp, i) => (
```

```
      <div className="card" key={i}>
```

```
        <div className="comp-header">
```

```
          <span className="comp-label">Comparable #{i + 1}</span>
```

```
          {comps.length > 1 && (
```

```
            <button className="btn-remove" onClick={() => removeComp(  
          )}
```

```
        </div>
```

```
      <div className="fields-grid" style={{ marginBottom: 14 }}>
```

```
        <div className="field" style={{ gridColumn: "span 2" }}>
```

```
          <label>Dirección</label>
```

```
          <input value={comp.direccion} onChange={e => updateComp(i
```

```
        </div>
```

```
      </div>
```

```
      <div className="fields-grid-3" style={{ marginBottom: 14 }}>
```

```
        <SelectInput label="Tipo" value={comp.tipo} onChange={v =>
```

```
        <SelectInput label="Estado" value={comp.estado} onChange={v
```

```

        <FieldInput label="Antigüedad (años)" value={comp.antiguedad}
      </div>
    <div className="fields-grid-3" style={{ marginBottom: 14 }}>
      <FieldInput label="M² totales" value={comp.m2_totales} onChange={updateComp} />
      <FieldInput label="M² cubiertos" value={comp.m2_cubiertos} />
      <FieldInput label="Ambientes" value={comp.ambientes} onChange={updateComp} />
    </div>
    <div className="fields-grid-3" style={{ marginBottom: 14 }}>
      <FieldInput label="Dormitorios" value={comp.dormitorios} onChange={updateComp} />
      <FieldInput label="Baños" value={comp.banos} onChange={updateComp} />
      <div className="field">
        <label>Precio</label>
        <div className="price-row">
          <select value={comp.moneda} onChange={e => updateComp(e)}>
            <option>USD</option>
            <option>ARS</option>
          </select>
          <input type="number" value={comp.precio} onChange={e => updateComp(e)} />
        </div>
      </div>
    </div>
    <div className="toggles-row">
      <Toggle label="Cochera" value={comp.cochera} onChange={v => updateComp(v)} />
      <Toggle label="Piscina" value={comp.piscina} onChange={v => updateComp(v)} />
    </div>
  </div>
  <div>
    <button className="btn-add" onClick={addComp}>+ Agregar otro comp</button>
  </div>
  </div>

  {error && <div className="error-box">{error}</div>}

  <div className="actions-row">
    <button className="btn-secondary" onClick={() => setStep(1)}>< Volver</button>
    {inputMode === "form" && (
      <button className="btn-primary" onClick={generateReport} disabled={loading}
        >{loading ? <span>Generando análisis...</span> : "Generar análisis"}</button>
    )}
  </div>
  {loading && (
    <div className="loading-overlay">
      <div className="spinner" />
      <p className="loading-text">La IA está analizando las propiedades</p>
    </div>
  )}

```

```

    </>
  })

  { /* STEP 3 */
  {step === 3 && report && (
    <>
    <p className="section-title" style={{ marginTop: 32 }}>Análisis Compa
    <p className="section-sub">{subject.direccion}</p>

    {sections.VALOR_ESTIMADO && (
      <div className="highlight-box">
        <div className="hl-label">Valor estimado de mercado</div>
        <div className="hl-value">{
          (() => {
            const t = sections.VALOR_ESTIMADO;
            const m = t.match(/USD[\s$]*([\d.,]+)\s*[--a]\s*USD[\s$]*([\d
            if (m) return `USD ${m[1]} - USD ${m[2]}`;
            const s = t.match(/USD[\s$]*([\d.,]+)/i);
            return s ? `USD ${s[1]}` : "Ver análisis";
          }) ()
        }</div>
      </div>
    )}

    {[
      { key: "RESUMEN", icon: "📄", label: "Resumen", color: "#1a2a3a" },
      { key: "TABLA_COMPARATIVA", icon: "📊", label: "Tabla comparativa",
      { key: "AJUSTES", icon: "⚖️", label: "Ajustes por diferencias", col
      { key: "VALOR_ESTIMADO", icon: "💰", label: "Valor estimado", color
      { key: "CONCLUSION", icon: "✅", label: "Conclusión y recomendación
    ].map(s => sections[s.key] && (
      <div className="report-section" key={s.key}>
        <div className="report-section-header">
          <div className="report-section-icon" style={{ background: s.col
            <span className="report-section-title">{s.label}</span>
          </div>
          <div className="report-section-body">{sections[s.key]}</div>
        </div>
      )})

    {!Object.keys(sections).length && (
      <div className="card">
        <div className="report-section-body">{report}</div>
      </div>
    )}

    <div className="actions-row" style={{ marginTop: 32 }}>

```



```
        <button className="btn-secondary" onClick={() => setStep(2)}>← Edit
        <button className="btn-primary" onClick={() => { setStep(1); setSub
            + Nuevo análisis
        </button>
    </div>
</>
    )}
</div>
</div>
);
}
```